VERIQTA

Build a Disaster Recovery Lab:

Back Up, Destroy, and Restore a Real Application



RTO / RPO Planning

Real Application Lab

Disaster Recovery Exercises

Verification & Recovery Runbook

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

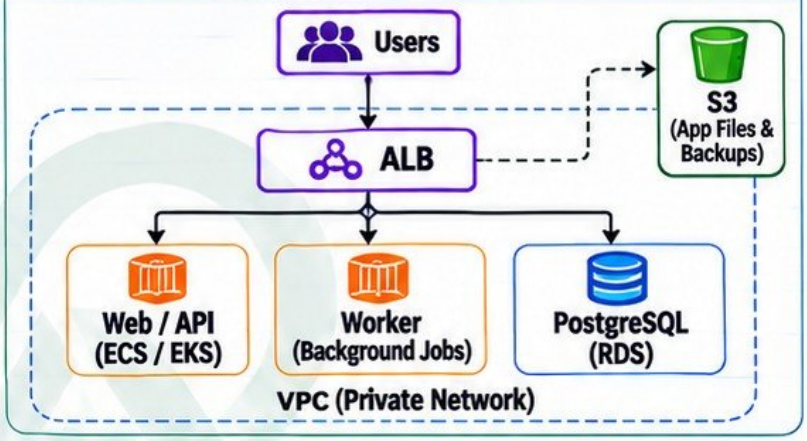
The Backup Exists. Can the Business Survive?

Define the application, architecture, failure boundaries, RTO, RPO, and recovery acceptance criteria.

1. Application Overview

- Order-processing application:
- Web/API for customers
- PostgreSQL for transactions
- Background workers (emails, jobs)
- File storage (uploaded receipts)
- Real business data (orders, payments)

2. High-Level Architecture



3. Failure Boundaries

- Single AZ or Service failure
- Database loss or corruption
- Accidental deletion (data or config)
- Regional disruption (DR scenario)
- Human error (misconfiguration)

4. Recovery Objectives

RTO (Recovery Time Objective)	**≤ 60 minutes** (restore and make the app usable)
RPO (Recovery Point Objective)	**≤ 5 minutes** (maximum acceptable data loss)

5. Recovery Acceptance Criteria

- Application is fully restored in a clean environment.
- Data is consistent and within the RPO (≤ 5 minutes).
- Core functions work (login, place order, view order, payment).
- Background jobs resume succcessfully.
- Uploaded files are available and correctly linked.
- Recovery time meets the RTO (≤ 60 minutes).

6. Key Takeaways

- A backup is only valuable if it can restore a working business.
- Define the application, risks, and targets (RTO/RPO) before backing up.
- Set clear and testable recovery acceptance criteria.

VERIQTA

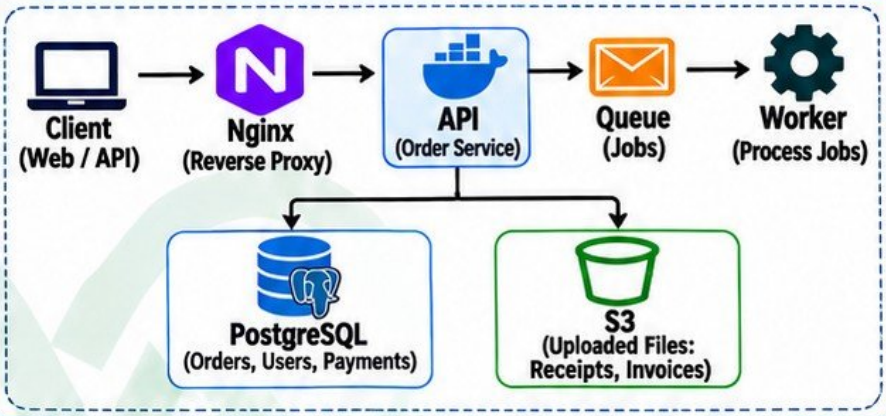
Build an Application Worth Recovering

Deploy a working order-processing API, PostgreSQL, worker, and file storage.

1. Project Goal

- Deploy a real, working order-processing application with these components:
  - API (web service)
  - PostgreSQL (database)
  - Worker (background jobs)
  - File storage (uploaded files)

2. Application Architecture



3. Tech Stack

- Docker (containerization)
- Nginx (reverse proxy)
- Python (FastAPI)
- PostgreSQL (database)
- Redis (queue / caching)
- AWS S3 (file storage)
- Docker Compose (deployment)

4. Step-by-Step Deployment

1. Create the project structure and application code.
2. Define the Dockerfiles and docker-compose.yml.
3. Build and start the services (API, database, worker).
4. Configure file storage (S3) and application settings.
5. Test the application end to end (place an order, upload a file, process a job).

5. Verify It Works

- API is reachable and healthy.
- Orders can be created and listed.
- Background jobs are processed.
- Files are uploaded and stored in S3.
- Data is persisted in PostgreSQL.

6. Expected Outcome

- A fully working order-processing application.
- Data stored in PostgreSQL.
- Background tasks processed by a worker.
- Files (e.g., receipts/invoices) stored in S3.
- A stable, containerized setup ready for backup and disaster recovery exercises.

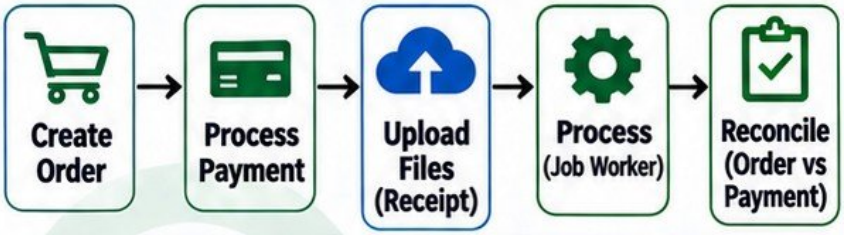
Create Data That Cannot Be Faked

Generate orders, payments, uploads, idempotency keys, and reconciliation records.

1. Objective

- Generate realistic, verifiable business data for the application.
- Cover the full order lifecycle: order → payment → upload → processiing → reconciliation.
- Ensure data integrity, traceability, and no duplicates (idempotency).

2. Data Flow (Order Lifecycle)



3. What to Generate

- **Orders** (e.g., 500+ orders with different statuses)
- **Payments** (successful, failed, and refunded)
- **Uploads** (receipt, invoice, or document files)
- **Idempotency Keys** (unique key per request to prevent duplicates)
- **Reconciliation Records** (match orders, payments, and files)

4. Example Data (Database)

order_id	amount	status	idempotency_key
ORD-1001	49.99	PAID	3f9a1e7b- ...
ORD-1002	120.00	PROCESSING	7b2d4c8e-...
ORD-1003	75.50	REFUNDED	d91e6a2c-...

5. How to Generate (Example)

```
$ python seed_data.py \
    --orders 500 \
    --payments \
    --uploads sample-files/ \
    --reconcile.
```

6. Verification Checks

- Orders, payments, and uploads are created.
- Each request has a unique idempotency key.
- Reconciliation matches payments to orders.
- No duplicate orders or payments exist.

7. Expected Outcome

- A rich, realistic dataset for testing backups and recovery.
- Data relationships are consistent (orders ↔ payments ↔ files).
- Ready for the disaster recovery exercises.

VERIQTA

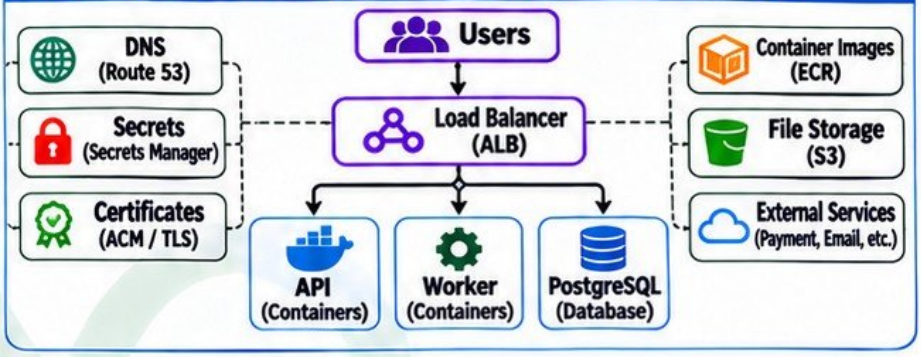
Map Every Hidden Recovery Dependency

Identify secrets, DNS, container images, certificates, network configuration, and external services.

1. Objective

- List and document all components the application depends on to run and recover.
- Identify hidden or indirect dependencies.
- Capture how each dependency is backed up and restored.

2. Dependency Map (Order Application)



3. Dependency Inventory

Dependency	Example	Recovery Source
Secrets	DB credentials, API keys	Backup / Export (Secrets Manager)
DNS	app.veriqta-lab.com (Route 53)	IaC / DNS backup (Zone export)
Container Images	api:v1.0, worker:v1.0 (ECR)	Image registry (Replicated)
Certificates	TLS for domain (ACM)	Reissue / Export (ACM)
Network Config	VPC, subnets, SGs, routes	IaC (Terraform)
External Services	Payment gateway, email (SES)	Reconfigure with stored credentials

4. How to Map (Steps)

1. Review the application architecture.
2. List all direct and indirect dependencies.
3. Record ownership, configuration, and location.
4. Identify the backup and restoration method for each dependency.
5. Validate by simulating a partial recovery (e.g., restore in a new environment).

5. Example: Dependency Record

Component	Details	Backup / Restore
PostgreSQL	RDS (prod)	Automated snapshots
S3 (Files)	order-receipts-bucket	Versioning + replication
ECR (Images)	api:v1.0, worker:v1.0	Keep image copies
Secrets	DB user, API keys	Export / restore (AWS SM)
DNS	Route 53 hosted zone	Zone backup + IaC
Certificates	ACM (TLS for domain)	Reissue from ACM
External API	Payment, email	Stored config + test keys

6. Expected Outcome

- A complete, documented dependency map for the application.
- All critical and hidden dependencies identified.
- Clear backup and restoration steps for each dependency.
- A repeatable checklist for future recovery exercises.

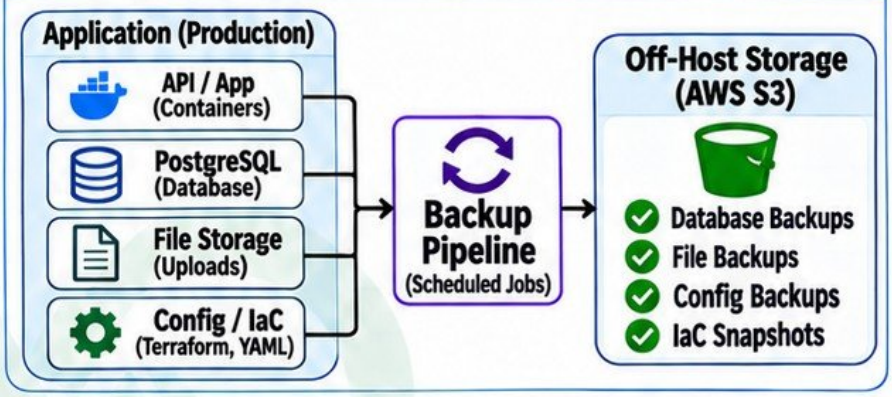
Design a Recoverable Backup Architecture

Implement database, file, configuration, and infrastructure backups with off-host storage.

1. Objective

- Design and implement a reliable, restorable backup architecture for the application.
- Back up database, files, configuration, and infrastructure.
- Store backups off-host (for example, in Amazon S3).

2. Backup Architecture (High-Level)



3. What to Back Up

Component	Method (Example)	Frequency
Database (PostgreSQL)	pg_dump / RDS snapshot	Every 15 min (or hourly)
Files (Uploads)	S3 sync (aws s3 sync)	Every hour
Configuration (App / System)	Backup config files, .env, Helm, YAML	Daily (on change)
Infrastructure (IaC)	Versioned in Git (+ state backup)	On change and daily

4. Implementation Steps

1. Create an S3 bucket for backups (with versioning).
2. Back up the PostgreSQL database (pg_dump or RDS snapshot).
3. Sync application files to S3 (e.g., uploads, receipts).
4. Back up configuration files and application settings.
5. Back up infrastructure code and Terraform state.
6. Automate with a scheduled job (e.g., cron or CI pipeline).

5. Backup Policies

- RPO: ≤ 5 minutes (maximum acceptable data loss).
- Retention: 30 days (daily), 12 months (monthly).
- Encryption: Enable encryption at rest (S3 / KMS).
- Immutability: Use S3 Object Lock (protect from deletion).

6. Expected Outcome

- A complete, automated backup system covering data, files, configuration, and IaC.
- Backups stored securely off-host in Amazon S3.
- Clear backup schedule, retention, and protection.
- Ready for restore testing and disaster recovery exercises.

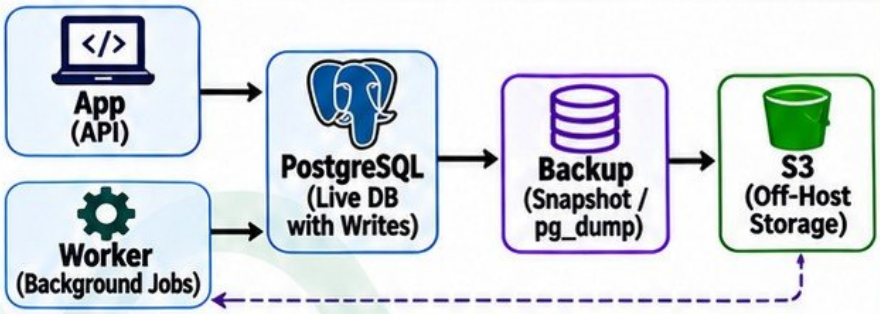
The Backup Completed but Was It Consistent?

Test backup consistency while writes and background jobs continue.

1. Objective

- Verify that the backup is consistent (not partial or corrupt) while the application remains live.
- Keep writes and background jobs running during backup.

2. Consistency Test Architecture



3. Test Procedure

1. Start continuous writes (create orders and process jobs).
2. Run the backup job (e.g., pg_dump or RDS snapshot).
3. Keep the application and worker running during the backup.
4. Restore the backup to a separate instance (test environment).
5. Compare the data between live and restored databases.

4. Consistency Verification

Check	Command / Method
Row Count Match	`SELECT COUNT(*) FROM orders;`
Data Integrity	Compare key records (e.g., latest orders/payments).
No Partial Writes	Verify all related tables (orders, payments, jobs) are in sync.
Job State	Confirm background jobs (queue/processed) are consistent.
Data Freshness	Check the last transaction timestamp matches the live system.

5. Example: Backup and Verify

```
# 1. Generate continuous writes
$ python seed_orders.py --run
# 2. Take a database backup
$ pg_dump -U appuser -d orders_db -f backup.sql
# 3. Restore to a test database
$ psql -U appuser -d orders_test -f backup.sql
# 4. Verify record count
$ psql -U appuser -d orders_db -c "SELECT COUNT(*)
  FROM orders;"
$ psql -U appuser -d orders_test -c "SELECT COUNT(*)
  FROM orders;"
```

6. Expected Outcome

- Backup is consistent and restorable while writes and jobs run.
- Row counts and key records match between live and restored databases.
- No partial or corrupt data.
- A repeatable test process to validate backup consistency.

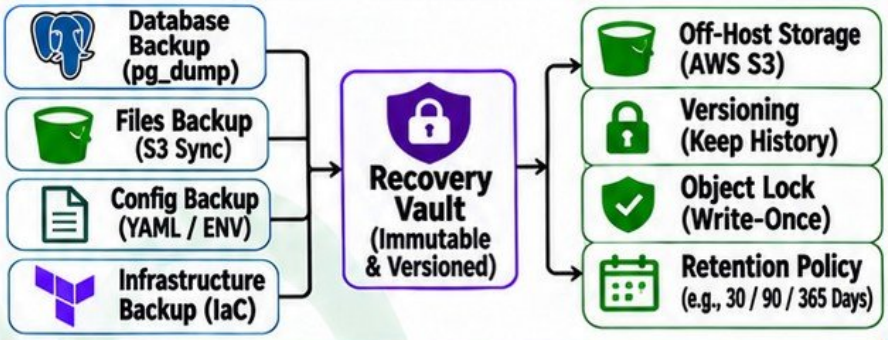
Build a Recovery Vault That Survives Operator Mistakes

Protect backup copies from accidental deletion, credential compromise, and retention failures.

1. Objective

- Build a secure, tamper-resistant recovery vault for backup copies.
- Prevent accidental deletion, unauthorized access, and retention failures.
- Ensure backups remain recoverable when needed.

2. Recovery Vault Architecture



3. Key Protections

Accidental Deletion	Use versioning and Object Lock (WORM).
Credential Compromise	Use least privilege (IAM) and dedicated backup roles.
Retention Failure	Set lifecycle rules and retention policies (e.g., 30/90/365).
Tampering	Enable immutable storage (Write-Once-Read-Many).
Single Point of Failure	Keep a copy in a separate account or region.

4. Implementation Steps

1. Create a dedicated backup storage (S3 bucket).
2. Enable versioning and Object Lock (immutable storage).
3. Apply a lifecycle and retention policy (e.g., keep daily for 30 days, monthly for 12 months).
4. Use a separate IAM role with least privilege.
5. Replicate backups to a second region/account.

5. Example: S3 Backup Vault (CLI)

```
# 1. Create the backup bucket
$ aws s3api create-bucket --bucket veriqta-dr-vault \
   --region us-east-1
# 2. Enable versioning
$ aws s3api put-bucket-versioning --bucket veriqta-dr-vault \
   --versioning-configuration Status=Enabled
# 3. Enable Object Lock (immutable)
$ aws s3api put-object-lock-configuration --bucket veriqta-dr-vault \
   --object-lock-configuration \
'ObjectLockEnabled="Enabled",Rule={DefaultRetention={Mode="COMPLIANCE",
Days=90}}'
```

6. Expected Outcome

- A protected, tamper-resistant recovery vault.
- Backups are versioned, immutable, and retained by policy.
- Backups remain safe from accidental deletion or credential compromise.
- Resilient and ready for disaster recovery.

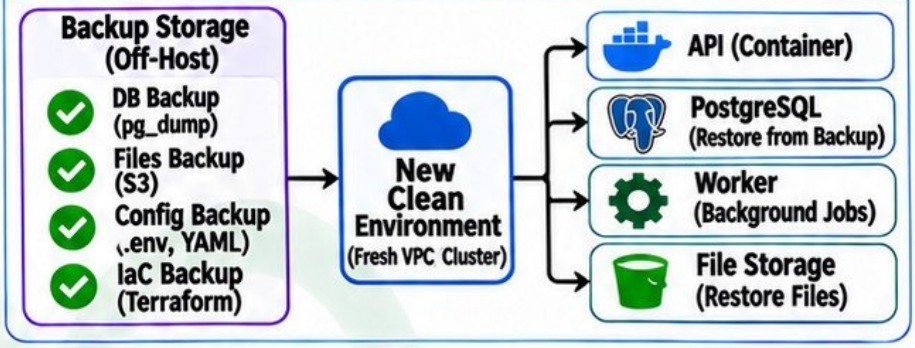
Restore Into a Clean Environment

Recover the complete application without relying on the original running environment.

1. Objective

- Restore the full application (API, database, worker, and files) in a new, clean environment.
- Use only the backup copies (no access to the original).
- Verify that the application works and data is intact.

2. Restore Architecture (New Environment)



3. Restore Steps (High Level)

1. Provision a new, clean environment (VPC or K8s cluster).
2. Deploy the application infrastructure (using IaC or manifests).
3. Restore the PostgreSQL database from backup.
4. Restore application files (uploads) from S3.
5. Apply configuration (secrets, .env, config maps).
6. Start the API and background workers.
7. Validate application functionality and data.

4. Key Commands (Example)

```
# 1. Create new environment (Terraform)
$ terraform init && terraform apply -auto-approve
# 2. Restore the database
$ pg_restore -U appuser -d orders_db \
  /backups/orders_db_2026-11-07.dump
# 3. Restore application files
$ aws s3 sync s3://veriqta-orders-backup/uploads/ \
  /app/uploads/
# 4. Deploy and start the application
$ docker compose up -d
# 5. Verify the application
$ curl http://<new-ip>/health
{"status"::"ok"}
```

5. Verification Checklist

- Database restored (correct row counts).
- Application starts and API is healthy.
- Files restored (uploads accessible).
- Background jobs are running (queue processed).
- Core business flows work (e.g., create order, view order, download file).

6. Expected Outcome

- A fully restored, working application in a new, clean environment.
- Data, files, and configuration recovered from backups.
- No dependency on the original environment.
- A repeatable, documented restore process (Runbook).

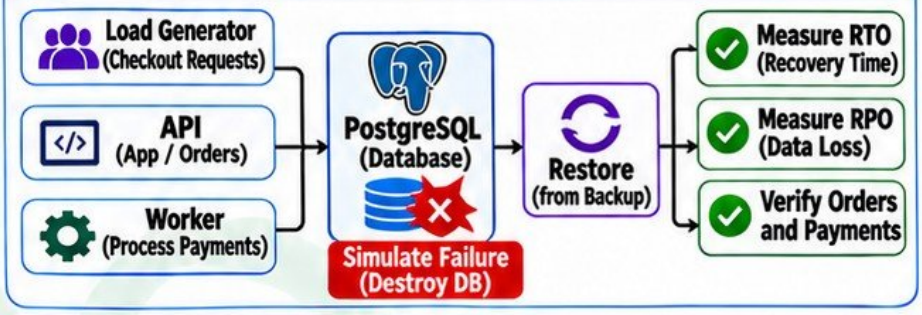
Destroy the Database During Checkout

Simulate database loss under active transaction load, then measure recovery and lost writes.

1. Objective

- Simulate a real database failure during active checkouts.
- Restore the database from backup.
- Measure recovery time (RTO) and lost writes (RPO).

2. Failure Test Architecture



3. Test Steps

1. Start the application (API, worker, database).
2. Run a checkout load (continuous orders).
3. Destroy the database (e.g., delete the volume or drop the DB).
4. Observe application behavior (errors, retries, queueing).
5. Restore the database from the latest backup.
6. Reconnect the application and workers.
7. Compare data and measure recovery and lost writes.

4. Key Commands (Example)

```
# 1. Run checkout load (in a separate terminal)
$ python load_checkout.py --rate 20
# 2. Simulate database loss (example: Docker)
$ docker volume rm orders_db_data
# 3. Restore from backup
$ pg_restore -U appuser -d orders_db \
  /backups/orders_db_latest.dump
# 4. Bring the application back
$ docker compose up -d
# 5. Check data and count orders
$ psql -U appuser -d orders_db -c "SELECT COUNT(*)
  FROM orders;"
```

5. Metrics to Capture

Metric	What It Tells You
RTO (Recovery Time)	Time from failure to full restore (e.g., 6 minutes).
RPO (Data Loss)	Maximum lost writes (e.g., up to 1 minute of orders).
Total Orders (Before)	Orders created before failure.
Total Orders (After)	Orders after restore.
Lost Writes	Orders missing after recovery (e.g., before: 10,000, after: 9,998, lost: 2).

6. Expected Outcome

- Database failure is successfully simulated during active checkouts.
- Application is restored from backup.
- RTO (recovery time) is measured and within target (e.g., ≤ 10 minutes).
- RPO (data loss) is known and acceptable (e.g., ≤ 1 minute).
- Orders and payments are verified and reconciled.

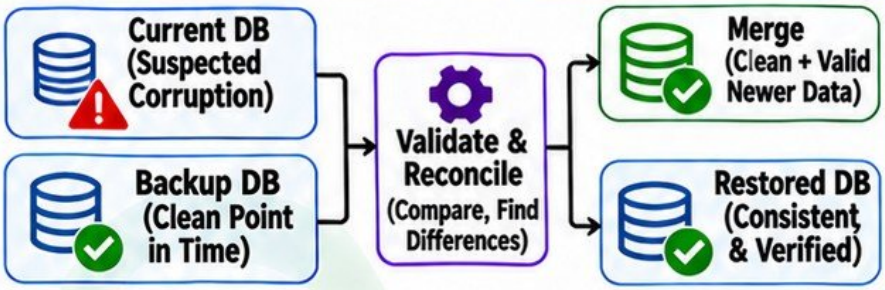
Recover From Silent Data Corruption

Inject incorrect records and recover a clean point without blindly overwriting valid newer data.

1. Objective

- Detect silent data corruption (wrong or inconsistent records).
- Recover to a clean, consistent state.
- Preserve valid newer data (no blind overwrite).

2. Recovery Strategy (High Level)



3. Corruption Simulation

Operation	Example
Incorrect Record	Order total = -100
Inconsistent Data	Payment marked "paid" but no matching order
Missing Reference	Order withut a valid customer_id
Stale Data	Old status overwrites a newer correct value

4. Recovery Steps

1. Take a clean, verified backup (point in time).
2. Analyze and detect corrupted records (validation rules and queries).
3. Create a reconciliation report (differences).
4. Merge only the affected records from the clean backup (do not overwrite the whole DB).
5. Keep valid newer data (e.g., orders created after the backup time).
6. Verify data integrity and application functionality.

5. Key Commands (Example)

```
# 1. Detect anomalies
$ psql -d orders_db -c "SELECT id, total
  FROM orders WHERE total < 0;"
# 2. Compare with clean backup
$ pg_dump -d orders_db --data-only --table=orders \
  > current.sql
$ pg_dump -d orders_db_clean --data-only --table=orders \
  > clean.sql
$ diff current.sql clean.sql | head
# 3. Merge corrected records (example)
$ psql -d orders_db -f reconcile_orders.sql
```

6. Expected Outcome

- Corrupted data is identified and corrected.
- A clean, consistent database is restored.
- Valid newer data is preserved (no full overwrite).
- Orders, payments, and related records remain accurate and in sync.
- Application works with verified, clean data.

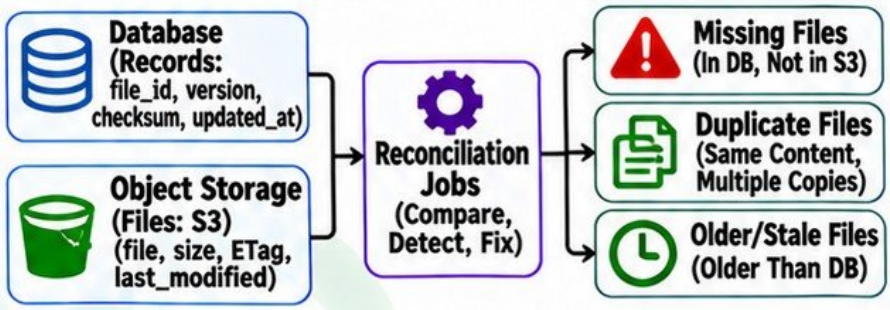
The Files Restored but the Database Disagrees

Reconcile database records against missing, duplicated, or older object-storage files.

1. Objective

- Align database records with files in object storage (e.g., S3).
- Detect missing files, duplicates, and older (stale) files.
- Reconcile data and restore consistency.

2. Reconciliation Architecture



3. Common Mismatches

Issue	Example
Missing File	DB has file_id F123, but no file in S3.
Duplicate File	Same file uploaded twice (F123, F123-copy).
Older File	S3 file last_modified is older than DB updated_at.
Version Mismatch	DB points to v2, but S3 still has v1.

4. Reconciliation Steps

1. List DB records and files from object storage.
2. Compare by file_id, checksum (ETag), and version.
3. Identify missing, duplicate, and older files.
4. Fix issues (restore missing, remove duplicates, replace older files with the latest).
5. Update database metadata (version, checksum, timestamps).
6. Re-run verification to confirm consistency.

5. Example Commands

```
# 1. List DB file records
$ psql -U appuser -d orders_db -c \
  "SELECT file_id, version, checksum, updated_at
  FROM files;"
# 2. List files in S3
$ aws s3 ls s3://veriqta-orders/uploads/ --recursive
# 3. Find records with missing files
$ python reconcile_files.py --check-missing
# 4. Remove dupliates and update metadata
$ python reconcile_files.py --fix --update-db
```

6. Expected Outcome

- Database and object storage are fully aligned.
- Missing files are restored.
- Duplicate files are removed or merged.
- Older files are replaced with the latest version.
- File metadata in the database is accurate and consistent.

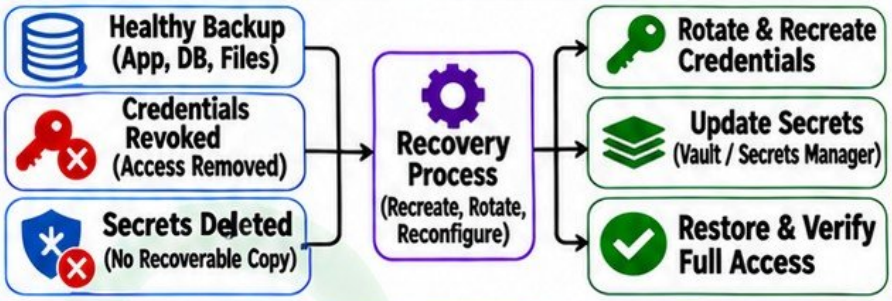
The Backup Is Healthy but the Credentials Are Gone

Recover after deliberately revoking lab credentials and removing recoverable secrets.

1. Objective

- Restore access and rebuild a working environment when credentials and secrets are lost.
- Use secure secret recovery and rotation.
- Verify the application works with new credentials.

2. Recovery Flow (High Level)



3. Lab Scenario (Simulated)

Backup Status	Healthy (recent and consistent).
Action Taken	Revoke access (IAM / DB / API keys) and delete secrets.
Impact	Application cannot connect (authentication fails).
Goal	Recover access, reconfigure secrets, and restore service.

4. Recovery Steps

1. Identify all required credentials and secrets (DB, API, storage, registry, etc.).
2. Recreate credentials with least privilege (new IAM roles, DB user, API keys).
3. Store secrets securely (e.g., AWS Secrets Manager, HashiCorp Vault, or Kubernetes Secrets).
4. Update the application configuration (env, config, K8s secrets) with the new secrets.
5. Restart services and verify end-to-end access.

5. Example Commands

```
# 1. Recreate a DB user and grant minimal access
$ psql -U postgres -c "CREATE USER appuser WITH
  PASSWORD 'NewP@ss123';"
$ psql -U postgres -c "GRANT CONNECT, SELECT, INSERT,
  UPDATE ON DATABASE orders_db TO appuser;"
# 2. Update secret (Kubernetes)
$ kubectl create secret generic db-credentials \
 --from-literal=username=appuser \
 --from-literal=password=NewP@ss123 --dry-run=client -o \
  yaml | kubecl apply -f -
# 3. Restart and verify
$ kubectl rollout restart deployment/api
$ curl http://api.veriqta.local/health
```

6. Expected Outcome

- Access is successfully restored with new, rotated credentials.
- Secrets are securely stored and managed.
- The application reconnects to the database and dependent services.
- Backup data remains intact and usable.
- The environment is secure and operational.

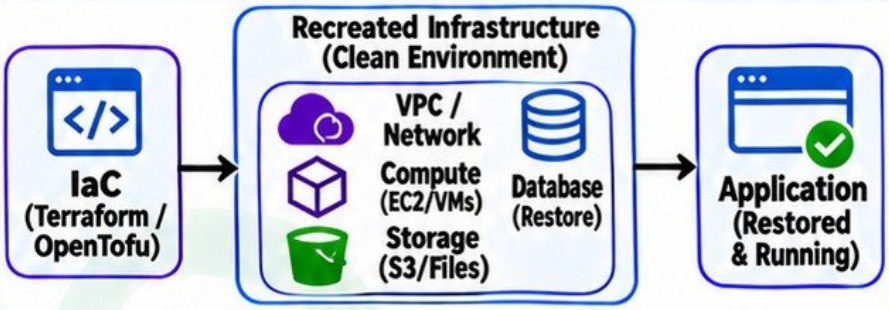
Rebuild After Total Infrastructure Loss

Destroy disposable application infrastructure and reconstruct it using infrastructure as code.

1. Objective

- Rebuild the entire application environment after a total loss.
- Use infrastructure as code (IaC) to recreate all resources.
- Restore application, data, and service access.

2. Architecture Overview



3. Recovery Steps

1. Confirm total infrastructure loss (resources destroyed).
2. Use IaC code (Terraform/OpenTofu) from version control (GitHub).
3. Recreate core infrastructure (VPC, subnet, security groups, compute, storage, database).
4. Restore data from off-host backup (DB and files).
5. Deploy the application (containers or VM).
6. Verify service and application health.

4. Key Commands (Example – Terraform)

```
# 1. Get the IaC code
$ git clone https://github.com/veriqta/iac-app.git
$ cd iac-app
# 2. Initialize Terraform
$ terraform init
# 3. Review what will be created
$ terraform plan
# 4. Recreate the infrastructure
$ terraform apply -auto-approve
# 5. Verify resources
$ terraform state list
```

5. Verification Checklist

- Infrastructure recreated from IaC (matches expected plan).
- Database restored and accessible.
- Application deployed and running.
- Files and object storage restored.
- Health checks and core business flows work (e.g., login, create order, list data).

6. Expected Outcome

- A fully reconstructed, working environment from code.
- Application, data, and storage restored.
- No manual, one-off configuration.
- Repeatable and consistent recovery process (IaC).
- Lower recovery time and reduced risk in future disasters.

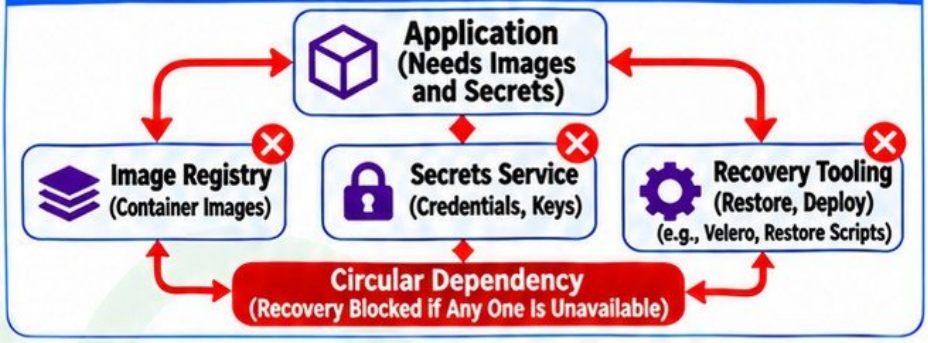
Your Recovery Process Has a Circular Dependency

Test what happens when the image registry, secrets service, or recovery tooling is unavailable.

1. Objective

- Validate recovery when a critical dependency is down (image registry, secrets service, or recovery tooling).
- Identify and remove circular dependencies.

2. Dependency Problem (Example)



3. Test Scenarios

Scenario	What to Test
Registry Down	Can you deploy using a local or cached image?
Secrets Service Down	Can you recover using a backup or offline secrets?
Recovery Tooling Down	Can you restore manually or with an alternative tool?
Circular Dependency	Can you break the loop and still recover?

4. Recovery Strategies

1. Break the dependency loop (deploy minimal foundation first).
2. Use a local or offline image registry mirror (e.g., Harbor).
3. Restore secrets from an encrypted backup (e.g., S3, Vault backup).
4. Keep a minimal, independent recovery toolchain (e.g., static binaries or container bundle).
5. Document a dependency-free recovery path (runbook).

5. Example Commands

```
# 1. Start a local registry (fallback)
$ docker run -d -p 5000:5000 --name registry registry:2
# 2. Load a backup image and push it locally
$ docker load -i app-backup.tar
$ docker tag app:latest localhost:5000/app:latest
$ docker push localhost:5000/app:latest
# 3. Restore secrets from backup
$ aws s3 cp s3://veriqta-backup/secrets.enc ./
$ sops -d secrets.enc > secrets.yaml
# 4. Deploy using the fallback resources
$ kubectl apply -f recovery/base.yaml
```

6. Expected Outcome

- Recovery is successful even when one or more dependencies are down.
- Circular dependencies are identified and removed.
- The application is restored using a fallback path (local images, backup secrets, independent tooling).
- A documented, repeatable recovery process is available.

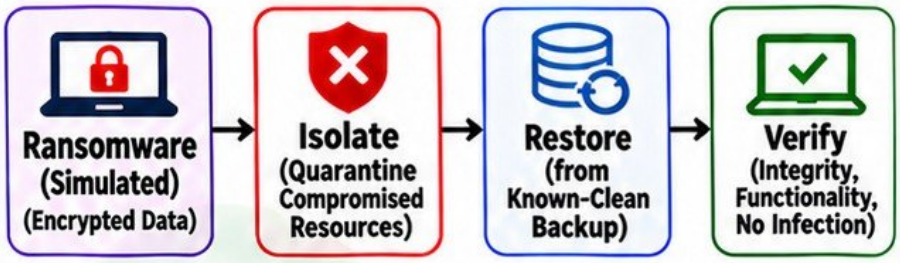
Simulate a Ransomware-Style Recovery Event

Isolate compromised lab components and restore from a known-clean, protected recovery point.

1. Objective

- Simulate a ransomware incident in the lab.
- Isolate affected components to stop further damage.
- Restore the application and data from a known-clean backup.

2. Incident & Recovery Flow



3. Ransomware Lab Scenario

Component	Action (Simulated)
Application	Files encrypted (e.g., *.locked, *.enc).
Database	Data made unreadable (simulated encryption).
Services	Some services fail or go offline.
Network	Isolate affected nodes to prevent spread.

4. Recovery Steps

1. Detect and confirm the ransomware simulation (encrypted files, service issues, alerts).
2. Isolate compromised components (e.g., remove from load balancer, block network access).
3. Validate a known-clean, immutable backup (e.g., versioned/S3 Object Lock, read-only copy).
4. Restore the application, database, and files to a clean environment.
5. Scan and verify the environment (no remaining infection) and bring services back online.

5. Example Commands (Linux)

```
# 1. Simulate ransomware (lab only)
$ find /app/data -type f -exec rename 's/$/.locked/' {} \;
# 2. Isolate the compromised node
$ kubectl cordon app-node-1
$ kubectl drain app-node-1 --ignore-daemonsets
# 3. Restore from a clean backup (example)
$ aws s3 sync s3://veriqta-backup/clean/ /app/data \\
  --exact-timestamps
# 4. Verify and restart
$ systemctl restart app
$ curl -I http://app.veriqta.local/health
```

6. Expected Outcome

- The ransomware simulation is contained and does not spread.
- The application and data are restored from a known-clean, protected backup.
- Services are back online and functioning correctly.
- A repeatable incident response and recovery process is documented.

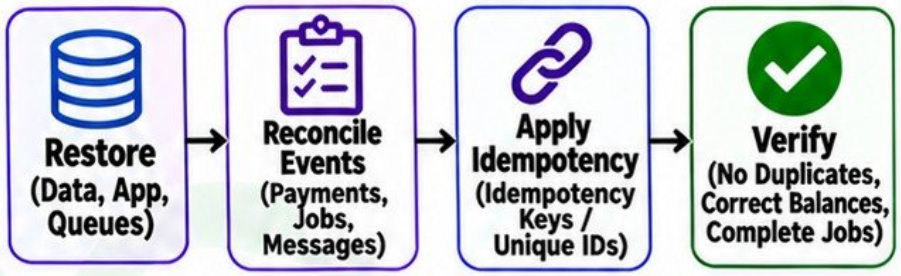
Recover Without Duplicating Customer Transactions

Reconcile payment events, queued jobs, duplicate messages, and idempotent writes after restoration.

1. Objective

- Restore the system without creating duplicate customer transactions.
- Reconcile payments, queued jobs, and replayed messages.
- Ensure idempotent processing and data consistency.

2. Recovery Flow (End-to-End)



3. Events to Reconcile

Event	What to Check
Payment Events	No duplicate charges (same transaction ID).
Queued Jobs	Jobs not executed twice (e.g., emails, fulfillment).
Messages	No duplicate processing (e.g., order.created).
Idempotent Writes	Safe to replay without creating new records.

4. Recovery Steps

1. Restore the application, database, and queues from a consistent backup.
2. Extract and compare event logs (payments, jobs, messages).
3. Use idempotency keys or unique transaction IDs to detect and ignore duplicates.
4. Reprocess only the missing or incomplete events.
5. Reconcile final state: orders, payments, job status, and customer balances.

5. Example Commands

```
# 1. Check for duplicate payments
$ psql -U appdb -d orders -c "SELECT
  transaction_id, COUNT(*) FROM payments
  GROUP BY transaction_id HAVING COUNT(*) > 1;"
# 2. Reprocess only failed or pending jobs
$ python replay_jobs.py --status=pending
# 3. Use idempotency key in an API request
$ curl -X POST https://api.veriqta.local/payments \
  -H "Idempotency-Key: txn_7f9a2c" \
  -d '{"order_id": "ORD123", "amount": 49.99}'
```

6. Expected Outcome

- All valid transactions are restored exactly once.
- No duplicate charges, jobs, or messages.
- Queues are in a clean, consistent state.
- Idempotent processing is in place for future recoveries.
- Customer data, orders, and balances are accurate and complete.

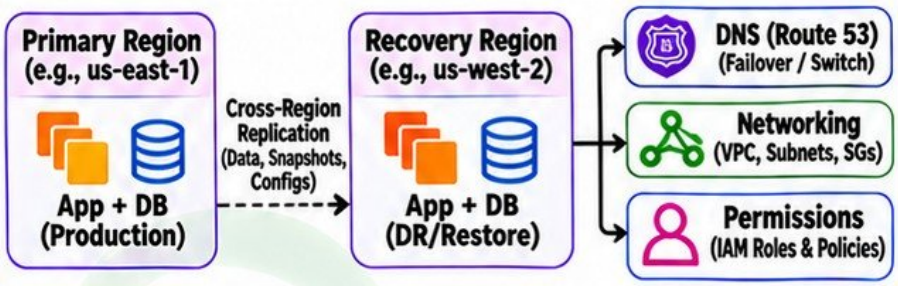
Run a Cross-Region Recovery Exercise

Restore in a second AWS Region and validate DNS, networking, permissions, and application functionality.

1. Objective

- Test disaster recovery by restoring the application in a second AWS Region.
- Validate DNS, networking, IAM permissions, and end-to-end functionality.
- Confirm the application is usable by real users.

2. Architecture Overview (AWS)



3. Recovery Test Plan

Area	What to Validate
DNS	Failover to the recovery Region (Route 53).
Networking	VPC, subnets, routing, and security groups work.
Permissions	IAM roles and policies provide correct access.
Application	App starts, connects to DB, and all key features work.

4. Recovery Steps

1. Replicate data and required resources to a second AWS Region (e.g., us-west-2).
2. Recreate infrastructure in the recovery Region (using IaC, e.g., Terraform/CloudFormation).
3. Restore the application and data (DB snapshot, S3 replication, or backup).
4. Update DNS (Route 53) to point to the recovery Region (failover).
5. Validate: connectivity, permissions, and full application functionality.

5. Example Commands (AWS CLI)

```
# 1. Copy an RDS snapshot to another Region
$ aws rds copy-db-snapshot \
    --source-db-snapshot-identifier prod-snap \
    --target-db-snapshot-identifier dr-snap \
    --source-region us-east-1 --region us-west-2
# 2. Sync application data to S3 (cross-region)
$ aws s3 sync s3://veriqta-app-prod/ \
    s3://veriqta-app-dr/ --region us-west-2
# 3. Update Route 53 to switch traffic
$ aws route53 change-resource-record-sets \
    --hosted-zone-id Z123456789 \
    --change-batch file://failover.json
```

6. Expected Outcome

- Application successfully restored in the second AWS Region.
- DNS resolves to the recovery Region.
- Networking and security (VPC, SGs, NACLs) work correctly.
- IAM permissions are validated and functional.
- Application is fully operational and accessible to users.

Automate the Disaster Recovery Runbook

Build scripted recovery with preflight checks, approvals, checkpoints, and verification gates.

1. Objective

- Automate the end-to-end disaster recovery process.
- Include preflight checks, approval controls, checkpoints, and verification gates.
- Ensure a repeatable, safe, and auditable recovery.

2. Automated Recovery Flow

Preflight Checks (Readiness & Health) → Approval (Human Gate (e.g., Slack/ Emaill) → Run Recovery (Script / IaC / Pipeline) → Checkpoints (Validate Each Stage) → Verification Gates (Confirm Full Recovery)

3. Runbook Stages

Stage	Purpose
Preflight Check	Verify prerequisites (e.g., backups, quota, health).
Approval	Get explicit go-ahead to start recovery.
Recovery Execution	Run automated scripts (e.g., Terraform, Ansible).
Checkpoint	Pause and validate key milestones (e.g., infra, data, app).
Verification Gate	Confirm full service health and business functionality.

4. Recovery Script Structure (Example)

```
dr-runbook/
├── preflight.sh     # System readiness checks
├── approve.sh       # Approval gate (Slack/Email)
├── recover.sh       # Restore infra, data, app
├── checkpoints.sh   # Validate each recovery stage
├── verify.sh        # Final verification (health)
└── runbook.sh       # Orchestrates the full flow
```

5. Example Commands

```
# 1. Run preflight checks
$ ./preflight.sh
# 2. Request approval (manual or via Slack)
$ ./approve.sh
# 3. Execute recovery (e.g., via Terraform)
$ ./recover.sh
# 4. Validate checkpoint (e.g., services alive)
$ ./checkpoints.sh
# 5. Final verification (health & functionality)
$ ./verify.sh
```

6. Expected Outcome

- A fully automated, controlled recovery process.
- Preflight, approval, checkpoints, and verification gates are enforced.
- Infrastructure, data, and application are restored and validated.
- Recovery is repeatable, auditable, and easy to run in future incidents.

Conduct a Blind Disaster Recovery Game Day

Give an engineer an unknown failure scenario and measure actual recovery performance.

1. Objective

- Run a realistic, blind Disaster Recovery (DR) exercise.
- Give the engineer an unknown failure (no prior details).
- Measure recovery time, accuracy, and overall effectiveness.

2. Game Day Flow

Unknown Failure (Surprise Scenario) → Engineer Investigates (Diagnose & Plan) → Recover (Service, Data, Access) → Validate (Health, Functionality, Data) → Measure & Review (RTO, Errors, Lessons)

3. Example Blind Scenarios

Scenario (Unknown to Engineer)	Impact
Primary database outage	App errors, no data access
Region or AZ failure	Service unavailable
Secrets/credentials revoked	Services fail to start
Corrupted or missing data	Inconsistent or incomplete data

4. Performance Metrics

1. RTO (Recovery Time Objective): How long it takes to restore service.
2. RPO (Recovery Point Objective): How much data (if any) was lost.
3. Recovery Success: Was the service fully restored?
4. Accuracy: Did the engineer follow correct and safe steps?
5. Lessons Learned: What worked, what failed, and what to improve.

5. Example Exercise Outline

```
# Game Day (High-Level Steps)
1. Start the exercise (engineer gets only an alert, e.g., "Production service is down").
2. Engineer investigates (logs, metrics, health).
3. Identify the root cause (unknown failure).
4. Execute recovery (restore, failover, or rebuild).
5. Validate the system (health checks, user test, data integrity).
6. Record and review results (RTO, RPO, actions, gaps, and improvements).
```

6. Expected Outcome

- The engineer successfully recovers the service with minimal downtime and data loss.
- Actual RTO and RPO are measured and documented.
- Gaps in tools, process, or skills are identified.
- A clear action plan is created to strengthen DR readiness.

Final Production Recovery Review

Present recovery evidence, RTO/RPO results, unresolved risks, cost tradeoffs, and a go/no-go decision.

1. Objective

- Review the full recovery exercise.
- Present evidence of restoration and service health.
- Compare actual RTO/RPO with targets, assess risks and costs, and make a go/no-go decision.

2. Recovery Review Workflow

Collect Evidence (Logs, Metrics, Screenshots) → Measure Results (RTO/RPO, Uptime, Data Integrity) → Assess Risks (Gaps, Vulnerabilities, Open Items) → Review Costs (Infrastructure, Data Transfer, Operations) → Decide (Go / No-Go) (With Actions)

3. Recovery Results (Example)

Metric	Target	Actual	Status
RTO (Recovery Time)	≤ 60 min	42 min	✓
RPO (Data Loss)	≤ 15 min	8 min	✓
Service Uptime (after recovery)	≥ 99%	99.95%	✓
Data Integrity (Validation)	100%	100%	✓

4. Key Findings

- Application and core services restored successfully.
- RTO and RPO met the defined targets.
- Data integrity validated (no corruption or loss).
- Unresolved risks: single points of failure in monitoring and manual approval delay.
- Cost tradeoff: cross-region recovery increases monthly cost by ~20–30%.

5. Evidence Summary

- System health checks (service up, healthy).
- Recovery logs and timestamps (RTO/RPO).
- Data validation results (no missing data).
- Network/DNS validation (routing and access ok).
- Screenshots or monitoring dashboards (post-recovery).

6. Final Decision

GO

- Restore is stable and meets targets.
- Risks are documented with mitigation plan.
- Cost impact is acceptable.

NO-GO

- Critical risks remain unresolved.
- RTO/RPO not met.
- Data integrity or key services not fully validated.